CSC2103 Data Structures | Lecture 2

Hash Tables

School of Computing (demo) | Week 2

Sample lecture deck compiled from Wikipedia (CC BY-SA 4.0) for a demo.
Not official material of any university. See LICENSE.md for sources and attribution.
Wikipedia articles used: Hash table, Hash function, Linear probing, Open addressing.

Learning outcomes

- Explain how a hash function maps keys to buckets
- Resolve collisions with separate chaining and with open addressing
- Trace insertions with linear probing and count probes
- Use the load factor to decide when to resize a table

The dictionary (map) ADT

- Operations: **insert(key, value)**, **search(key)**, **delete(key)**

Implementation	Search	Insert
Unsorted array	O(n)	O(1)
Sorted array	O(log n)	O(n)
Balanced binary search tree	O(log n)	O(log n)
Hash table	O(1) average	O(1) average

Hash table: the idea

- An array of **m buckets** (slots)
- A **hash function** turns a key into an index: `index = hash(key) mod m`
- Example: m = 10, h(k) = k mod 10
  - 23 → bucket 3
  - 47 → bucket 7
  - 81 → bucket 1
- If every key gets its own bucket, search is a single array access

What makes a good hash function

- **Deterministic**: the same key always gives the same index
- **Uniform**: spreads keys evenly over the buckets
- **Fast** to compute, and uses the whole key
- Division method: $h(k) = k \bmod m$; choose m prime, not close to a power of 2
- Multiplication method: $h(k) = \lfloor m \cdot (kA \bmod 1) \rfloor$ with $0 < A < 1$ (Knuth suggests $A \approx (\sqrt{5} - 1)/2 \approx 0.618$)
- Strings: polynomial hash, for example $h = h \cdot 31 + \text{code}(c)$ for each character

Collisions

- A **collision**: two different keys hash to the same bucket
- Unavoidable when there are more possible keys than buckets (pigeonhole principle)
- Likely even when the table is mostly empty (the birthday problem)
- So every hash table needs a **collision resolution** strategy:
  - **Separate chaining**: each bucket holds a list of entries
  - **Open addressing**: probe other slots in the same array

Load factor

- **Load factor** $\alpha = n / m$ (n entries, m buckets)
- As α grows, collisions grow and operations slow down
- Typical thresholds: about 0.6 to 0.75 for open addressing; between 1 and 3 for separate chaining
- Java's `HashMap` resizes when α exceeds its default load factor of 0.75
- **Resize (rehash)**: allocate a bigger table (often double) and re-insert every entry: O(n), but O(1) amortised per insert

Separate chaining

- Each bucket points to a linked list (chain) of the entries that hash there
- Insert: hash, then add to the chain: O(1)
- Search and delete: hash, then scan one chain: **O(1 + α)** expected
- α may exceed 1: chains simply get longer
- Worst case: every key lands in one chain, so search is **O(n)**

Chaining example

- m = 7, h(k) = k mod 7
- Insert 50, 700, 76, 85, 92, 73, 101 (new entries go to the end of the chain)
- 50 → 1, 700 → 0, 76 → 6, 85 → 1, 92 → 1, 73 → 3, 101 → 3

Bucket	Chain
0	700
1	50 → 85 → 92
2	(empty)
3	73 → 101
4	(empty)
5	(empty)
6	76

Open addressing

- Every entry is stored **in the bucket array itself**; needs $\alpha \leq 1$
- On a collision, follow a **probe sequence** h(k, 0), h(k, 1), h(k, 2), … until an empty slot
- **Linear probing**: $h(k, i) = (h(k) + i) \bmod m$
- **Quadratic probing**: $h(k, i) = (h(k) + c_1 i + c_2 i^2) \bmod m$
- **Double hashing**: $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$, where $h_2(k)$ is never 0

Linear probing example

- m = 10, h(k) = k mod 10
- Insert 89, 18, 49, 58, 69
- 49 → 9 taken, try 0: **2 probes**
- 58 → 8, 9, 0 taken, try 1: **4 probes**
- 69 → 9, 0, 1 taken, try 2: **4 probes**
- **Primary clustering**: occupied runs grow and attract more collisions

Slot	Key
0	49
1	58
2	69
3 to 7	(empty)
8	18
9	89

Expected probes with linear probing

- Knuth's estimates for a table with load factor α:
  - successful search ≈ ½ (1 + 1/(1 − α))
  - unsuccessful search (or insert) ≈ ½ (1 + 1/(1 − α)²)

Load factor α	Successful	Unsuccessful
0.50	1.5	2.5
0.75	2.5	8.5
0.90	5.5	50.5

Probes explode as α approaches 1: keep α well below 1 with open addressing.

Chaining vs open addressing

	Separate chaining	Open addressing
Where entries live	lists outside the array	inside the array
Load factor	can exceed 1	must stay below 1
Deletion	simple: remove from the list	needs a tombstone ("deleted" marker) so probe sequences stay intact
Cache behaviour	pointer chasing	good (contiguous probes)
Average search / insert / delete	O(1)	O(1)
Worst case	O(n)	O(n)

A chained hash map in JavaScript

```
class HashMap {
  constructor(m = 8) { this.buckets = Array.from({ length: m }, () => []); this.size = 0; }
  index(key) {
    let h = 0;
    for (const ch of String(key)) h = (h * 31 + ch.charCodeAt(0)) >>> 0;
    return h % this.buckets.length;
  }
  get(key) {
    for (const [k, v] of this.buckets[this.index(key)]) if (k === key) return v;
    return undefined;
  }
  set(key, value) {
    const chain = this.buckets[this.index(key)];
    for (const e of chain) if (e[0] === key) { e[1] = value; return; }
    chain.push([key, value]);
    if (++this.size / this.buckets.length > 0.75) this.resize();
  }
  resize() {
    const entries = this.buckets.flat();
    this.buckets = Array.from({ length: this.buckets.length * 2 }, () => []);
    for (const [k, v] of entries) this.buckets[this.index(k)].push([k, v]);
  }
}
```

Summary and tutorial

- Hash tables give O(1) average search, insert and delete; O(n) in the worst case
- Collisions: chaining (lists per bucket) or open addressing (probe the array)
- Keep the load factor low and resize by doubling

Tutorial 2: (1) Insert 10, 22, 31, 4, 15, 28, 17, 88, 59 into a table of size 11 with h(k) = k mod 11 using linear probing, then using chaining. (2) Why must $h_2(k)$ never be 0 in double hashing? (3) Compute the expected unsuccessful-search probes for $\alpha = 0.8$.

References and licence

- Wikipedia, *Hash table*, https://en.wikipedia.org/wiki/Hash_table (accessed 10 Oct 2026)
- Wikipedia, *Hash function*, https://en.wikipedia.org/wiki/Hash_function (accessed 10 Oct 2026)
- Wikipedia, *Linear probing*, https://en.wikipedia.org/wiki/Linear_probing (accessed 10 Oct 2026)
- Wikipedia, *Open addressing*, https://en.wikipedia.org/wiki/Open_addressing (accessed 10 Oct 2026)

Text adapted and condensed from the articles above, licensed CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0/). This deck is shared under the same licence. Code snippets written for this deck.